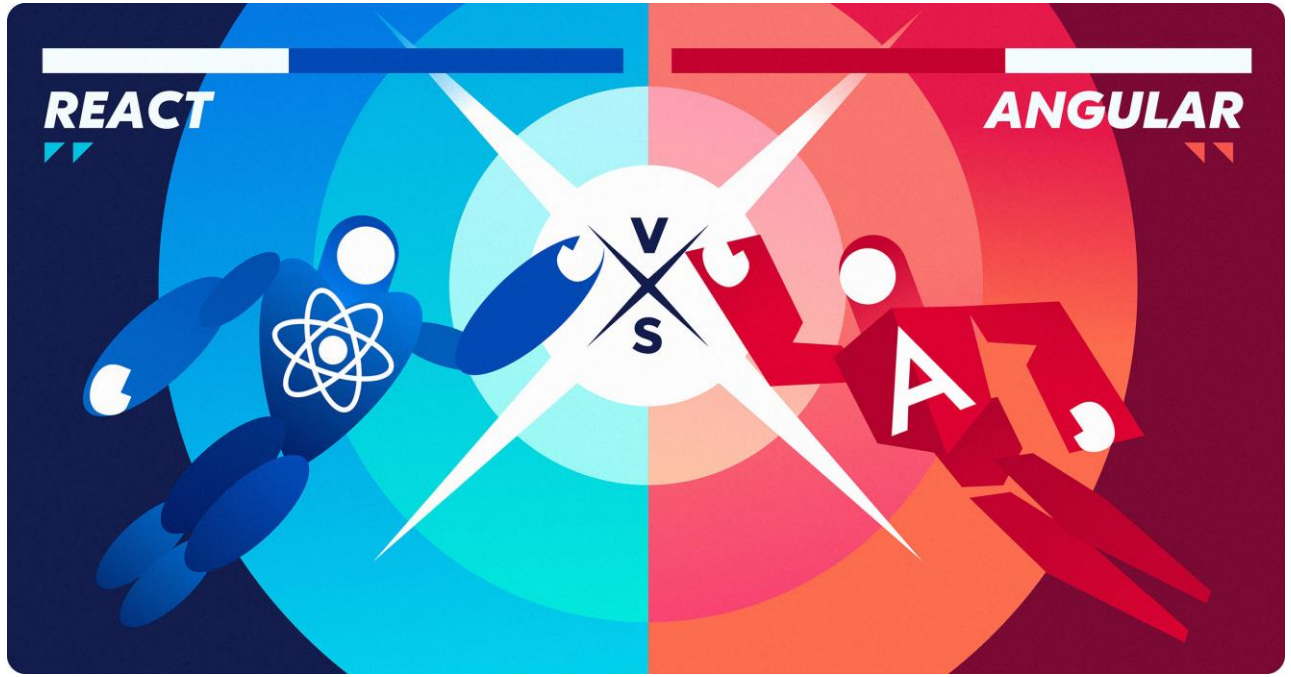


TECHNOLOGIES WEB II (React & Angular)

Professeur	Dr. Fadwa LACHHAB
Niveau	4eme année
Année	2024-2025

Partie 2 : Angular

Semaine 1	Semaine 2	Semaine 3	Semaine 4	Semaine 5	Semaine 6
Presentation de Angular Environnement de développement Components	Data Binding/Directives	Forms/ Services Routing	Interaction avec Base de Données : FireBase	Pipes /HTTP/Observables Authentification	Examen TP

Angular

Formulaire dans Angular : Forms

Forms

- Dans **Angular** à partir de la version 2, voir jusqu'à la version 7 et même après, on peut distinguer deux types de création des formulaires.
- **Template-Driven Forms :** souvent est la méthode la plus utilisée pour la création et la gestion des formulaires.
- Cette approche est appelée l'approche de **AngularJS**. Car c'est la même utilisée dans AngularJS. Cela ne signifie pas qu'elle est moins bonne.
- **Reactive Forms:** C'est la moins utilisées mais la plus façonnable et flexible.

Template-Driven Forms

- Cette approche se base essentiellement sur deux directives **“ngModel”** et **“ngForm”.**
- **ngForm**

Elle est utilisée dans votre balise HTML **<form ngForm=“...”></form>** pour exporter les valeurs saisies et les états **“ngForm”** des validateurs vers la partie background.

Donc, à partir du component, si on veut chercher qu’est ce qu’un user a saisi donc on récupère **“ngForm”** et on cherche dans cette collection les valeurs des inputs souhaitées.

- **ngModel**

Cette directive est utilisée à l'intérieur de la balise HTML **<input ngModel=“”></input>** .

Elle est utilisée pour exposer la valeur de cet input ou même autre control. Donc son rôle est d’enregistrer ce champ en utilisant l’attribut **“name”.**

Template-Driven Forms

- Dans cet exemple, on va essayer de créer un formulaire de contact qui permet de collecter quelques information **(Nom, Email et adresse),** avec un bouton submit et un autre bouton Reset.

TS contact.component.ts ×

src > app > contact > TS contact.component.ts > ...

```
3   @Component({
4     selector: 'app-contact',
5     templateUrl: './contact.component.html',
6     styleUrls: ['./contact.component.css'],
7   })
8   export class ContactComponent implements OnInit {
9     constructor() {}
10    ngOnInit(): void {}
11    submittedData: any = null;
12    onFormSubmit(userForm: NgForm): void {
13      this.submittedData = userForm.value;
14      console.log('Form Data:', userForm.value);
15    }
16    resetUserForm(userForm: NgForm): void {
17      userForm.resetForm();
18    }
19  }
```


UNIVERSIAPOLIS Université Internationale d'Agadir الجامعة الدولية لأكادير
ECOLE POLYTECHNIQUE D'AGADIR

Template-Driven Forms

Etape 1:

- On commence par créer la balise **<form>.**
- on passe la directive **“ngForm”** dans une variable qu’on l’a nommée **#userForm**.
- Cette variable **#userForm** sera utilisée comme paramètre de la fonction **‘onFormSubmit’** qu’on a déjà créé dans la définition du component.

contact.component.html

act.component.html > div.container > div.row > div.col-md-8.offset-md-2 > form > div.f

```
1   <div class="container">
2       <div class="row">
3         <div class="col-md-8 offset-md-2">
4           <form #userForm="ngForm" (ngSubmit)="onFormSubmit(userForm)">
5             <div class="form-group">
6               <label for="UserName">Nom</label>
7               <input
8                 type="text"
9                 id="UserName"
10                name="UserName"
11                ngModel
12                class="form-control"
13                placeholder="Entrez votre nom"
14              />
15            </div>
```

Template-Driven Forms

Etape 2:

- pour chaque input, on ajoute la directive **“ngModel”** et un attribut **“name”.**
- Attention avec **“ngModel”,** on n’utilise pas ni le symbole [] ni le symbole ().
- Si vous n’ajouter pas **ngModel** à votre champ ou input , ce champs ne sera plus pris en compte.

contact.component.html

act.component.html > div.container > div.row > div.col-md-8.offset-md-2 > form > div.f

```
1   <div class="container">
2       <div class="row">
3         <div class="col-md-8 offset-md-2">
4           <form #userForm="ngForm" (ngSubmit)="onFormSubmit(userForm)">
5             <div class="form-group">
6               <label for="UserName">Nom</label>
7               <input
8                 type="text"
9                 id="UserName"
10                name="UserName"
11                ngModel
12                class="form-control"
13                placeholder="Entrez votre nom"
14              />
15            </div>
```

Template-Driven Forms

- Le bouton **submit** n'a pas besoin d'une nouvelle fonction particulière car on a déjà défini la fonction **"onFormSubmit(...)"**.
- Aussi, dans les navigateurs modernes les boutons sont de type submit.
- Sinon, vous pouvez ajouter type="**submit**".
- Pour le bouton **Reset**, on ajoute une fonction pour l'événement (**click**)

nt.html ×

contact.component.html > div.container > div.row > div.col-md-8.offset

```
<div class="form-group text-center">
  <button type="submit" class="btn btn-primary">Submit</button>
  <button
    type="button"
    class="btn btn-danger"
    (click)="resetUserForm(userForm)"
  >
    Reset
  </button>
</div>
</form>
```

Template-Driven Forms

Lorsque vous remplissez le formulaire et cliquez sur **Submit**, les informations saisies s'affichent en bas du formulaire. Lorsque vous cliquez sur **Reset**, les champs du formulaire et les résultats affichés sont réinitialisés.

contact.component.html

src > app > contact > contact.component.html > div.container > div.row > div.col-md-8.offset

```
48        </form>
49        <!-- Résultats affichés ici -->
50      <div *ngIf="submittedData" class="mt-4">
51        <h4>Résultats soumis :</h4>
52        <p><strong>Nom :</strong> {{ submittedData.UserName }}</p>
53        <p><strong>Email :</strong> {{ submittedData.UserEmail }}</p>
54        <p><strong>Adresse :</strong> {{ submittedData.UserAddress }}</p>
55      </div>
```

Atelier Pratique

Nom

Entrez votre nom

Email

Entrez votre email

Adresse

Entrez votre adresse

Submit Reset

Nom

lachhab

Email

fadwa

Adresse

fadwa@gmail.com

Submit Reset

Résultats soumis :

Nom : lachhab

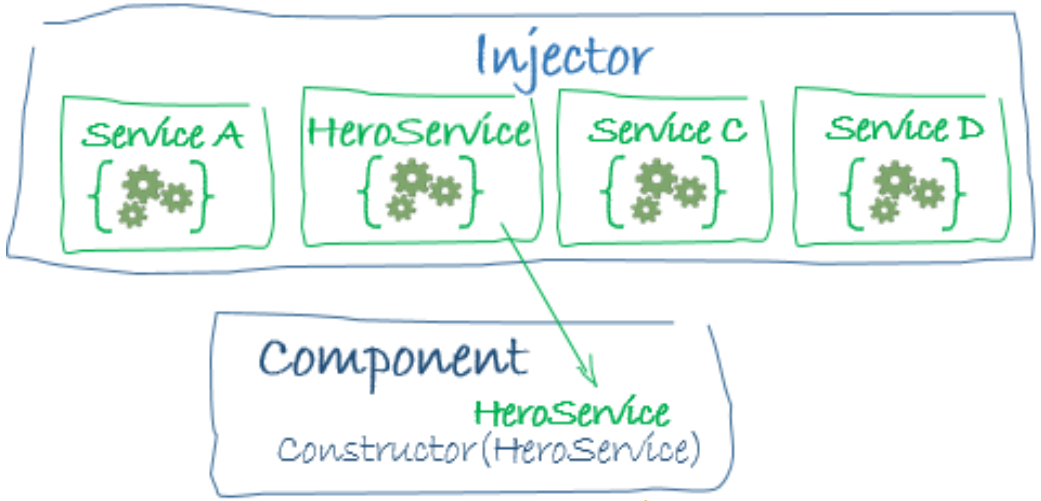
Email : fadwa

Adresse : fadwa@gmail.com

Services

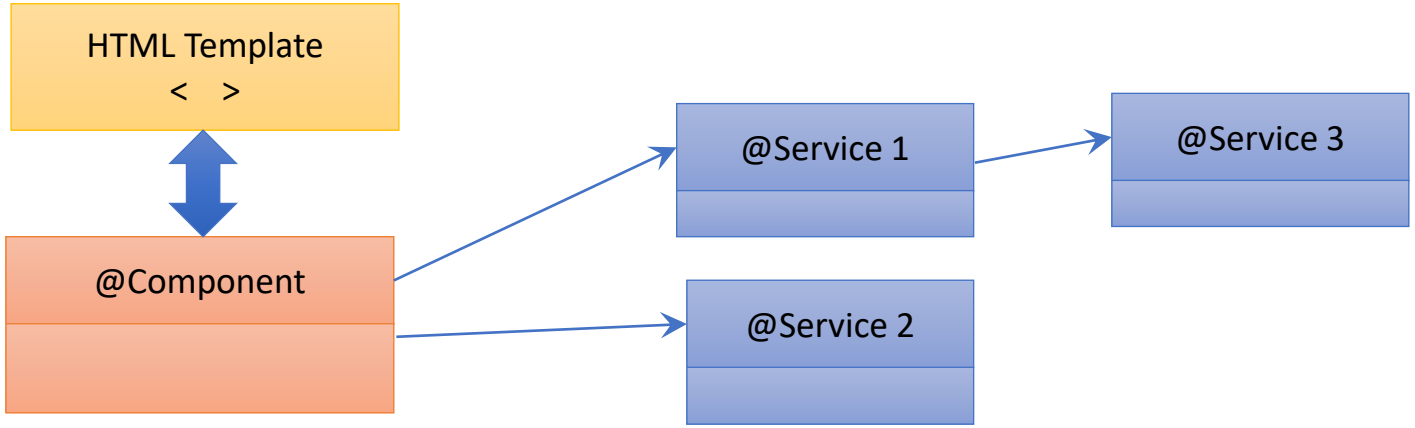
Services

- Un **service** est une **classe** qui permet d'exécuter un **traitement**.
- Permet **d'encapsuler** des **fonctionnalités** redondantes permettant ainsi d'éviter la **redondance de code**.
- Un service est associé à un composant en utilisant l'injection de dépendance
- Un service peut donc :
  - Interagir avec les données (fournit, supprime et modifie)
  - Interaction entre classes et composants
  - Tout traitement métier (calcul, tri, extraction …)



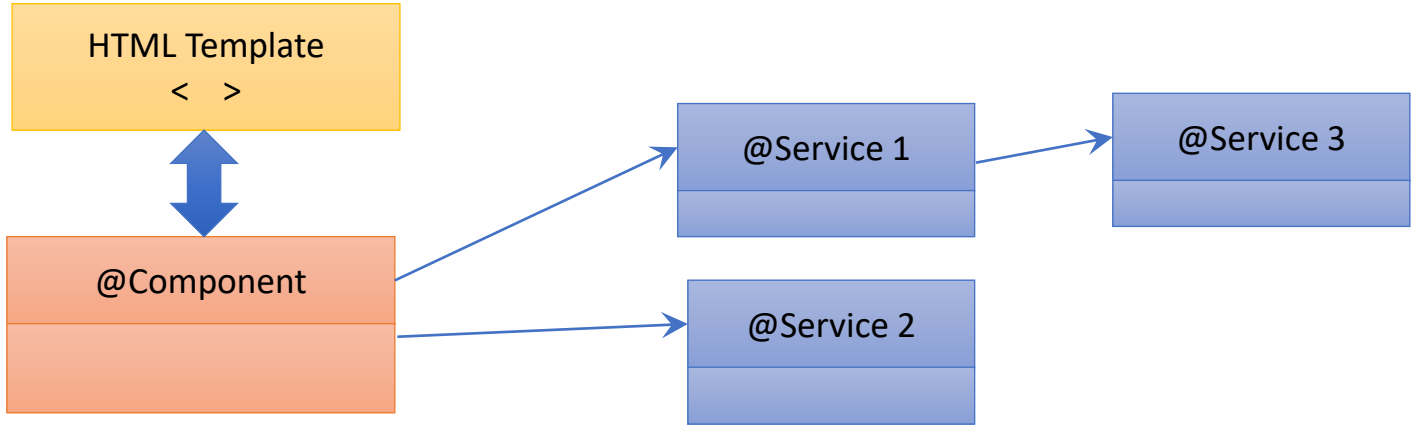
Services

- Un **service** est une catégorie large qui englobe toute valeur, fonction ou fonctionnalité dont votre application a besoin.
- Un service est généralement une **classe** avec un but étroit et bien défini.
- Généralement, les composants se limite à l'affichage et à la gestion des événements utilisateurs dans la vue du composant. L'exécution des **traitements en local** ou en **back end** sont attribués aux services.



Services

- Quand un **événement** survient dans la vue, le composant fait appel à des **fonctions** dans les **services** pour effectuer des **traitements** et fournir des résultats.
- Généralement, c'est les **services** qui interagissent avec la partie back end de l'application en envoyant des **requêtes HTTP**.
- Généralement c'est les composants qui consomment les **services**, toutefois, un service peut consommer d'autres services.
- l'utilisation d'un service se fait via le principe de l'injection des dépendances.



Services

- Dans **Angular**, un **service** est créé à l'aide du **Angular CLI** ou manuellement en créant une classe TypeScript. Les services sont utilisés pour encapsuler des fonctionnalités réutilisables, comme les appels API, la gestion des données ou le partage d'état entre composants.
- Vous pouvez créer un service avec la commande suivante dans le terminal :

```
ng generate service service-name
```

- ou en abrégé :

```
ng g s service-name
```

Services

- Par exemple, pour créer un service nommé donnees
- Cela génère deux fichiers :
  - donnees.service.ts : Le fichier contenant la logique du service.
  - donnees.service.spec.ts : Le fichier pour les tests unitaires.

```
PS C:\Users\Administrator\Documents\AngularApps\CoursPart3> ng g s donnees
CREATE src/app/donnees.service.spec.ts (362 bytes)
CREATE src/app/donnees.service.ts (136 bytes)
PS C:\Users\Administrator\Documents\AngularApps\CoursPart3>
```

Services

- [**providedIn: 'root'**]Fournit le service au niveau global
- Si vous ne voulez pas utiliser providedIn: 'root', vous pouvez enregistrer manuellement le service dans le module.

donnes.service.ts

```
1   import { Injectable } from '@angular/core';
2   @Injectable({
3     providedIn: 'root'
4   })
5   export class DonneesService {
6     constructor() { }
7     // Méthode pour récupérer des données
8     obtenirDonnees(): string {
9       return 'Voici les données fournies par le service';
10    }
11  }
```

TS app.module.ts × | <> app.component.html

src > app > TS app.module.ts > ...

```
14    ],
15    imports: [
16      BrowserModule,
17      AppRoutingModule,
18      FormsModule
19    ],
20    providers: [DonneesService],
21    bootstrap: [AppComponent]
22  })
23  export class AppModule { }
```

Enregistrement des services

- Pour utiliser un **service**, il faut préalablement enregistrer un **fournisseur** de ce service avec **l'injecteur**.
- Un **fournisseur** de service est une fabrique qui permet de gérer **l'instanciation des services**.
- Vous pouvez enregistrer les fournisseurs en **modules** ou en **composants**.
- En général, ajoutez des fournisseurs au module racine afin que la même instance d'un service soit disponible partout.

app.module.ts

```
imports: [
  BrowserModule, FormsModule
],
providers: [AboutService, DonneesService],
bootstrap: [AppComponent]
```

Enregistrement des services

Sinon, enregistrez-vous à un niveau de composant dans la propriété des fournisseurs des métadonnées **@Component**. Dans ce cas le service est instancié pour chaque nouvelle instance du composant.

```
@Component({
 selector: 'app-about',
 templateUrl: './about.component.html',
 styleUrls: ['./about.component.css'],
 providers: [DonneesService]
})
```

Injection d'un service dans un composant

Injection dans un composant : Importez le service et utilisez-le dans le constructeur pour accéder à ses fonctionnalités.

TS app.module.ts | <> app.component.html | TS app.component.ts ×

src > app > TS app.component.ts > AppComponent > ngOnInit

```
1  import { Component } from '@angular/core';
2  import { DonneesService } from './donnees.service';
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.component.html',
6    styleUrls: ['./app.component.css']
7  })
8  export class AppComponent {
9    constructor(private donneesService: DonneesService) {}
```

Injection des dépendances

- L'injection de dépendance est une façon de fournir une nouvelle instance d'une classe avec les dépendances entièrement formées dont elle a besoin.
- La plupart des dépendances sont des services.

TS donnees.service.ts ×

src > app > TS donnees.service.ts > DonneesService

```
1   import { Injectable } from '@angular/core';
2   @Injectable({
3     providedIn: 'root'
4   })
5   export class DonneesService {
6     constructor() { }
7     // Méthode pour récupérer des données
8     obtenirDonnees(): string {
9       return 'Voici les données fournies par le service';
10    }
11  }
```

TS donnees.service.ts TS app.component.ts ×

src > app > TS app.component.ts > AppComponent > constructor

```
1   import { Component } from '@angular/core';
2   import { DonneesService } from './donnees.service';
3   @Component({
4     selector: 'app-root',
5     templateUrl: './app.component.html',
6     styleUrls: ['./app.component.css']
7   })
8   export class AppComponent {
9     constructor(private donneesService: DonneesService) {}
10    donnees: string=''; // Propriété pour stocker les données
11    ngOnInit(): void {
12      // Appel au service pour obtenir les données
13      this.donnees = this.donneesService.obtenirDonnees();
14    }
15  }
```

Injection des dépendances

- L'injection de dépendance est une façon de fournir une nouvelle instance d'une classe avec les dépendances entièrement formées dont elle a besoin.
- La plupart des dépendances sont des services.

```
<hr/>
    <h1>Lancement du Service de donnes</h1>
    <p>{{ donnees }}</p>
```

Lancement du Service de donnes

Voici les données fournies par le service

UNIVERSIAPOLIS Université Internationale d'Agadir الجامعة الدولية لأكادير | ECOLE POLYTECHNIQUE D'AGADIR

Routage et Navigation
Routing

Routing

- **Angular** a donc besoin d'un système de routage pour permettre la **navigation** à travers différentes pages de votre
- application. Pour réaliser ce routage, Angular propose le module **RouterModule** disponible dans la librairie **@angular/router**.
- Une **route** est un **objet**.
- Les deux propriétés essentielles sont **path** et **component**.
- path permet de spécifier **l'URI**. Cette url ne doit pas commencer par un /
- component permet de spécifier le **composant** à exécuter.

Routage et Navigation

- Le **routeur** angulaire permet la navigation d'une vue à l'autre lorsque les utilisateurs exécutent des tâches d'application.
- Le routeur angulaire est un service facultatif qui présente une vue de composant particulière pour une URL donnée.
- Importez ce dont vous avez besoin comme vous le feriez à partir de tout autre paquet Angular.

src/app /app.module.ts (import)

```
import { RouterModule, Routes } from '@angular/router';
```

Configuration des routes

TS app-routing.module.ts ●

src > app > TS app-routing.module.ts > AppRoutingModule

```
1   import { NgModule } from '@angular/core';
2   import { RouterModule, Routes } from '@angular/router';
3   import { ContactComponent } from './contact/contact.component';
4   import { TraiterdataComponent } from './traiterdata/traiterdata.component';
5
6
7   const routes: Routes = [
8     { path: 'Acceuil', component: TraiterdataComponent }, // Route pour l'accueil
9     { path: 'contact', component: ContactComponent }, // Route pour la page "À propos"
10  ];
11
12  @NgModule({
13    imports: [RouterModule.forRoot(routes)],
14    exports: [RouterModule]
15  })
16  export class AppRoutingModule { }
```

routerLink

- Etant donnée cette configuration,
  - Quand l'utilisateur tape : http://localhost:4200/contact
  - le routeur cherche et charge le composant ContactComponent et l'affiche dans un élément **<router-outlet> </router-outlet>** .
  - Cet élément est sensé se trouver dans la vue du composant racine.

app.component.html

TS app-routing.module.ts × <> app.component.html ×

src > app > <> app.component.html > ...

```
1  <div class="container spacer">
2    <div class="container">
3      <button routerLink="/"  class="btn btn-primary">Acceuil</button>
4      <button routerLink="/contact" class="btn btn-primary">Contact</button>
5    </div>
6  <div class="container spacer">
7  <router-outlet></router-outlet>
8  </div>
9  </div>
```

Routage et Navigation

- Il est également possible de naviguer entre les différentes routes en utilisant la méthode **navigate()** du service Router. Pour cela le service Router doit être injecté dans la classe composant.
- L'exemple suivant montre le code de la classe du composant AppComponent : app.component.ts

```
import { Component } from '@angular/core';
import {Router} from "@angular/router";

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'app';
  constructor(private router:Router){}
  onAbout(){
    this.router.navigate(['contact']);
  }
}
```

```
<button (click)="onContact()"  class="btn btn-primary">>>> Aller a Contact</button>
```